

A Chatroom, Independent Attempts, and One Agent on Three Tasks

[Maintainer name and affiliation — to be filled before submission]

Corresponding author: [maintainer email]

Abstract

We compare a single agent, a three-seat chatroom, and selection from independent attempts on machine-scored tasks at roughly matched mean spend. All results are dated measurements of `claude-sonnet-5` under the `sonnet` alias on recorded subscription accounts, not estimates for other models or deployments. In Study 1, the chatroom passed both interpreter tasks on every seed, against 61/80 single-agent passes across their shared task family. The vote arm passed 75/80; direct tests did not distinguish it from the chatroom. On a numeric-formatting task, one agent passed 14/40, the chatroom 6/40, and the vote 7/40. The lower chatroom rate was significant under the pre-registered test but not a later Fisher test. Voting selected the common wrong implementation even though correct attempts were often available. The vote prediction succeeded on one interpreter task but failed its stated significance clause on the other.

The same recorded alias and settings yielded sharply different thinking-token distributions across subscription accounts. Study 2 mostly ran on the long-thinking account, leaving its pre-registered short-thinking primary comparisons essentially unobserved. Its small, exploratory long-thinking comparisons distinguished no arm from one agent. The comparison shows why serving-account provenance belongs with each run’s outcome.

A post-hoc audit asked what the system’s verification consisted of. Of 243 verification entries across 28 rooms, none ran agents on a changed build. In four audited rooms, re-running the author’s test was the most common review method and the least likely to find a defect. The verification rule accepts exactly that re-run.

1 Introduction

A chatroom can give several language-model agents a chance to challenge and revise one another’s work. It also buys more attempts at the task. To assess what the conversation contributes, the relevant comparison is with independent attempts chosen without seeing the answer, at a similar cost. This paper makes that comparison on a small, deliberately difficult task suite, then shows how a change in the serving account limited a later, broader study.

The measured system is `agent-chatroom-mcp`. Every benchmark run reported here used `claude-sonnet-5` under the `sonnet` alias between 18 and 20 September 2026. The subscription account active at run time is reconstructed from a sanitized switch log. Pass rates, costs, and which task was near ceiling belong to those dates, model and accounts. They are not predictions for newer models. The study method and the need to retain serving provenance do carry forward; the observed minority-versus-majority pattern is a hypothesis to test elsewhere.

Study 1 used seeds 101–140 on three fixed tasks, comparing one agent (A), a three-seat chatroom (C), and independent attempts with oracle-free selection (K). The third task asks for exact numeric formatting. All outcomes came from a machine oracle, not from the agents’ own verdicts.

Study 2 used seeds 501–520 and added a higher-effort single agent (AH) and a builder–reviewer pair (B). Its pre-registered primary comparison targeted short thinking. The run stopped with

missing cells after most work was served by the long-thinking account. The resulting long-thinking comparisons were specified after data had been seen and remain exploratory.

A review audit, added after both studies, examines what one seat’s check of another seat’s work consisted of, across 28 rooms run with a mix of models (Section 4.4).

The paper makes four narrow points. In Study 1, direct tests did not distinguish the chatroom from independent selection, even though both could outperform one agent on the interpreter family and both had lower observed success on printf. Across recorded accounts, thinking tokens differed under the same alias and settings, with one long-thinking single agent near ceiling on the measured interpreter task. Study 2 cannot answer its primary question: the intended short-thinking stratum was almost absent. Finally, the system’s verification rule certifies that an author’s evidence reproduces, and seats verified accordingly: re-runs dominated, and no verification entry ran agents on a changed build.

2 Evaluated system

The evaluated build of `agent-chatroom-mcp` gave each chatroom seat an independent opening before shared discussion. Seats could propose and amend one answer, challenge a quoted span, and vote on the current proposal. Agreement votes had to quote the proposal; the room used a supermajority rule that required unanimity with three active seats. A verification gate required a machine-parseable verdict from a connection other than the proposer, and the hub assigned that reviewer. The verify gate and reviewer assignment were present before both studies. Kick, replace and heartbeat changes dated 22 September were later additions and were not evaluated here. Study 1 chatroom cells record hub entry hash `66f9d135...`; run artifacts carry the revision used for each cell.

Arm A used one agent on the same public task brief. Arm K ran fresh, non-communicating A-style attempts and selected one answer without access to the hidden oracle. For the two interpreter tasks, it chose the most frequent normalized final answer, breaking a tie by sub-seed. For printf, it ran candidate programs on a fixed public-specification probe set and chose the implementation with the greatest execution agreement. Arm B in Study 2 passed one builder’s output to a reviewer for possible revision; AH used one agent at higher effort. Thus B was run, while Study 1 has no B cell.

3 Method

Tasks and scoring. Study 1’s tasks were admitted after a single-agent pilot screened about 45 designs for a discriminating pass rate. The admitted tasks each contain one known trap. `stamp-interpreter` and `stamp-2` ask for programs implementing the same invented-language scoping rule; they are two fixed instances of one family, not independent evidence from two domains. `bench-printf-format` asks for exact formatting where a natural implementation using the host runtime’s shortest representation disagrees with the specified output. The same hidden, machine-checked fixtures scored every arm. The seeds vary attempts on fixed task instances, rather than sampling new tasks. The task suite therefore supports comparisons on these traps, not an estimate across software engineering work.

The pilot targeted single-agent pass rates between 0.3 and 0.7, inclusive. The interpreter admission pilots used ten seeds each; the retained, raw-backed printf v1 pilot used fifteen seeds; v2 used ten, and five earlier v1 seeds lacked retained artifacts. Two versions of the printf task were piloted, and the one used here was chosen after both pilots’ results had been seen, a post-data selection.

Study 1 design. For each task, A and C ran on seeds 101–140, interleaved per seed. A’s budget ceiling was tied to C’s realized spend; matching did not force the same realized spend. K was separately pre-registered after A and C results but before K ran, on the same seeds and tasks. Its number of attempts was chosen from the earlier mean A and C costs, with each attempt run afresh. The selector had no oracle input. The fixed K group sizes were ten attempts for `stamp-interpreter`, eight for `stamp-2`, and seven for `printf`. The per-attempt cap left room for an ordinary A-style attempt; total group cost included every attempt even if it was killed or not selected. The matching target was mean realized C spend, not identical spend on every seed.

K ran from 13:22 to 15:33 UTC, before the account switch recorded at 16:38 UTC; an earlier shift-time estimate was superseded by that switch log. The comparison between K and C still spans separate collection windows. K exceeded its paired C spend on 19, 10, and 9 of the 40 seeds for the three tasks, despite matching at the mean.

The original A–C protocol specified a two-proportion z-test, using Fisher’s exact test for small cells, separately by task. After the data, the two interpreter tasks were pooled as a family and A–C was retested with Fisher’s exact test and Holm correction across the two families. We report that revised family result alongside the original per-task tests. The K pre-registration fixed six two-sided Fisher comparisons, K against A and C on each task, with Holm correction across all six. Pooled K comparisons on the interpreter family are exploratory and do not replace those registered tests.

The common seed labels put arms on the same task brief, but the attempts are separate model runs; the Fisher calculations do not treat them as paired observations.

Study 2 design and missingness. Study 2 specified five arms on seeds 501–520: A, AH, B, K and C. Its primary tests used the short-thinking stratum. The account switcher was not known to the harness at launch. A sanitized log later associated each run with its active account; mixed-account seeds were separated from within-regime comparisons. The run stopped at 190/200 cells, leaving ten absent. `Printf` seed 501 is kept in descriptive totals but excluded from inference after a documented cap deviation. One `printf` chatroom run has unknown cost. The long-thinking tests were added after much of that stratum had been observed, and use 19 seeds per arm on `stamp-interpreter` and 15 on `printf`. They use Fisher tests with Holm correction across the registered comparison family of fourteen comparisons, but their timing makes them exploratory. Strata condition on A’s realized post-run thinking, rather than an independent pre-run measurement; its sentinel also cannot rule out a change later in the same seed.

Account measurement. The account analysis joins single-seat run intervals to switch times and uses only runs with thinking telemetry for the thinking-token comparison. Long thinking was defined in the Study 2 pre-registration as at least 4,000 thinking tokens. Five of 1,947 eligible runs straddled a switch and were excluded. Output-token distributions give a second descriptive account comparison, but output across a three-seat room is not directly comparable to a single seat. Neither the active account nor effort was randomized. The client records the association, not the server-side reason for it.

4 Results

4.1 Study 1: conversation and independent selection

Table 1 gives the A–C family comparison; Figures 1 to 3 show the same Study 1 results as charts. Across the two interpreter programs, A passed 61/80 and C passed 80/80. The family-pooled

Table 1: Study 1, A versus C by task family.

Family	A pass/ <i>n</i>	C pass/ <i>n</i>	Fisher <i>p</i> (raw)	Holm <i>p</i> (<i>m</i> = 2)	Sig.
interpreter (stamp-interpreter + stamp-2)	61/80	80/80	0.000001	0.000002	yes
printf (bench-printf-format)	14/40	6/40	0.069165	0.069165	no

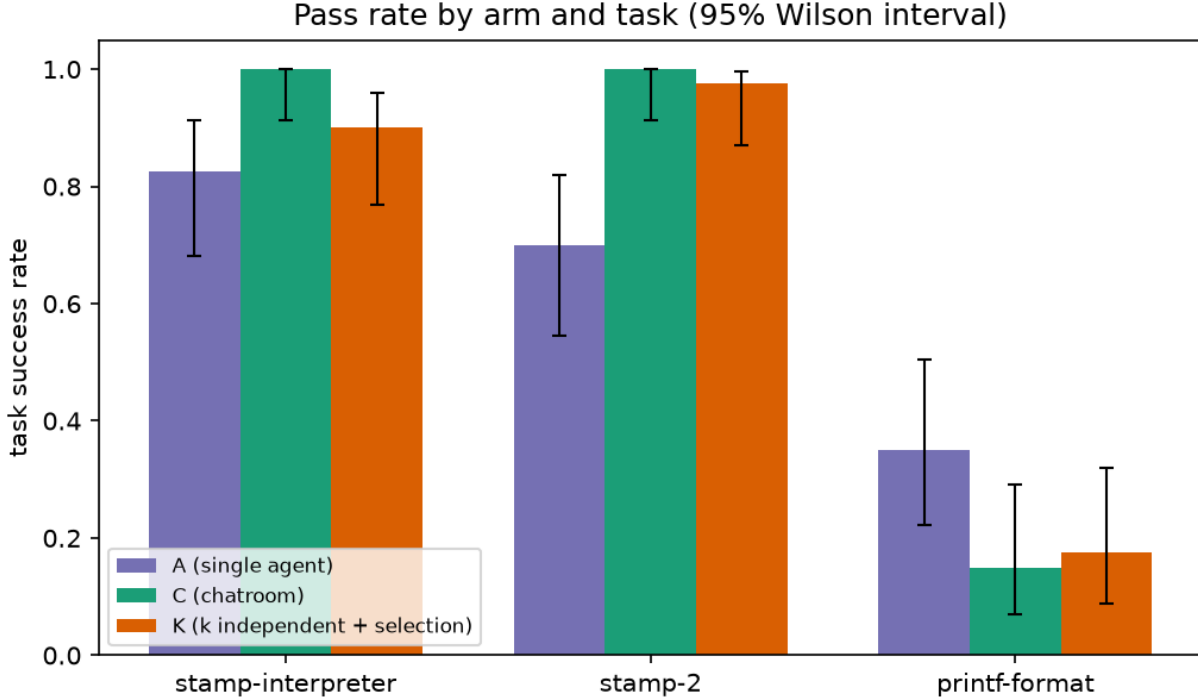


Figure 1: Study 1 task success by arm and task, with Wilson intervals at the 95 per cent level; 40 seeds per cell, short-thinking account, 19 September 2026. The chatroom (C) passed 40/40 on both interpreter tasks; on printf both aggregation arms fell below one agent (A).

Fisher test, adopted after the runs, gives $p = 0.000001$ before Holm adjustment. The original pre-registered per-task tests also rejected A–C on each interpreter program; the pooled result is a revised analysis, not the pre-registered test. K passed 36/40 on **stamp-interpreter** and 39/40 on **stamp-2**, or 75/80 pooled. Its pooled comparison with A ($p = 0.0033$) is exploratory. The registered K tests show a narrower result: K beat A on **stamp-2** (Holm $p = 0.009$), but was not distinguished from A on **stamp-interpreter** (Holm $p = 1.000$). The pre-registered prediction said K would beat A on both. That clause failed.

Direct K–C tests did not distinguish the arms on any Study 1 task: every Holm-adjusted p was at least 0.578. Pooling the interpreter tasks gives 75/80 for K and 80/80 for C, with an exploratory Fisher $p = 0.059$. Neither result establishes equality. In particular, a significant C–A result and a nonsignificant K–A result on **stamp-interpreter** are not themselves a test of C against K. The direct comparison is the relevant test, and the available sample leaves a small difference unresolved.

On **bench-printf-format**, A passed 14/40, C 6/40, and K 7/40. The pre-registered A–C z-test gave $p = 0.039$; the later family-based Fisher test gave $p = 0.069$, so the significance decision depends on the stated test. K was not distinguished from A (Holm $p = 0.578$) or C. The observed

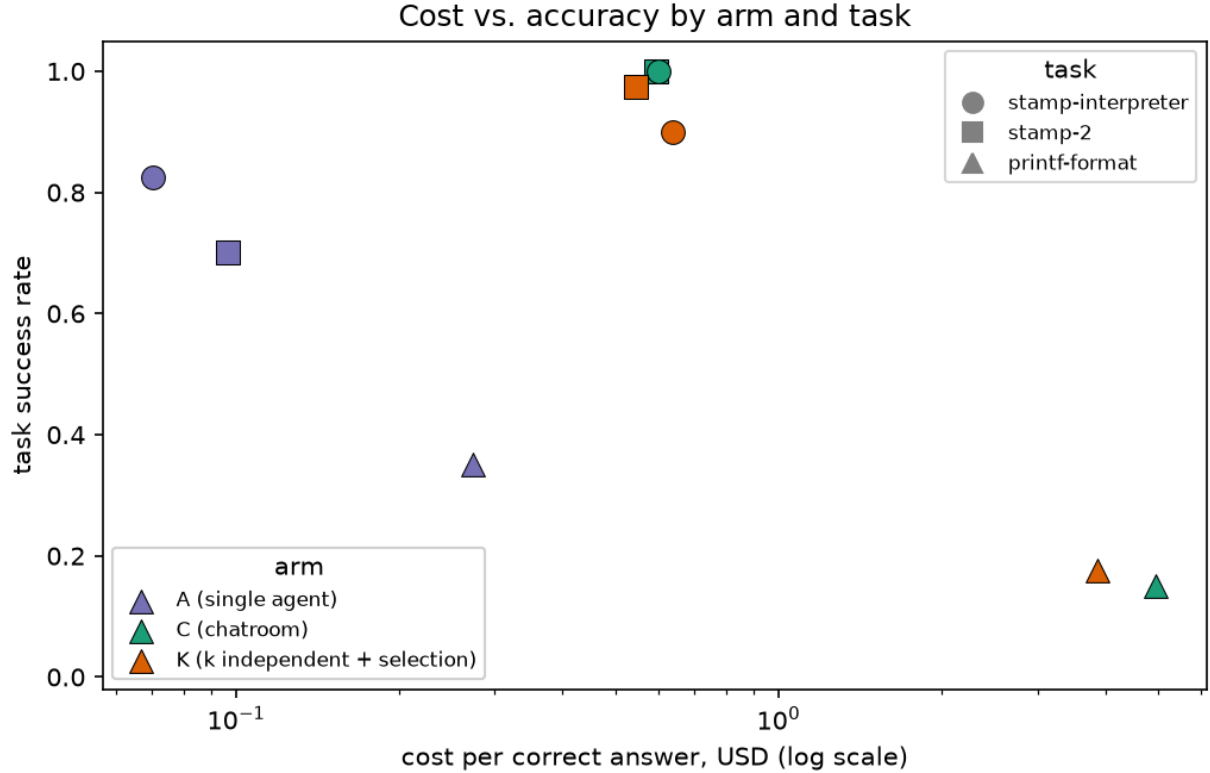


Figure 2: Study 1 cost per correct answer (log scale, API list price) against success rate, one point per task and arm. One agent was the cheapest per correct answer on every task; the chatroom and the vote cost about the same as each other on the interpreter tasks, and the vote cost less on printf (\$3.87 against \$4.97).

rates on this task are lower for both aggregation arms; they do not support a general assertion that either method lowers success on other tasks.

The independent-attempt evidence explains K’s printf outcome more specifically. In a fixed pool of 40 A attempts, all 14 passing implementations agreed on every one of 9,650 public-specification probes, and all 26 failing implementations also agreed with one another. Thus execution agreement identified the larger wrong cluster. In the real K groups, at least one correct attempt was available in 37/40, but the oracle-free selector submitted a correct one in only 7/40. The availability count is an oracle ceiling, not a deployable arm. The C artifacts retain pass or fail, but not the failing implementations; the same modal-error explanation for C is an association, not an observed C selection mechanism.

The interpreter answers show the opposite population pattern in Study 1. For **stamp-interpreter**, the seven A failures among 40 were identical; for **stamp-2**, 11 of the 12 failures were identical and one differed. Here the correct answer was the majority of A answers in the observed pool. The room summaries add diagnostic context: at least 26 of 40 **stamp-interpreter** rooms and 14 of 40 **stamp-2** rooms reported an erroneous opening followed by another seat’s correction. These are manual lower bounds from final seat summaries, without retained raw room events. They do not identify a causal chatroom mechanic.

The extra computation had a substantial price in these runs. C’s cost per correct answer relative

Arm K mechanism: does agreement track correctness?

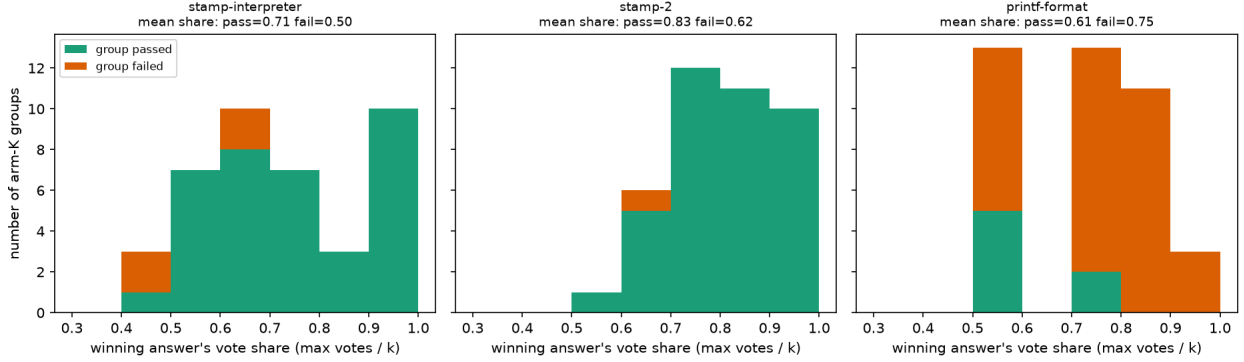


Figure 3: Study 1 vote arm: in each group, the winning answer’s share of the votes, split by whether the selected attempt passed. On printf, strong agreement goes with failure, because the failing implementation was the larger cluster of independent attempts.

Table 2: Recorded subscription account and run-level thinking or output tokens. Generated from the per-run records joined to the account-switch log.

Active account	Runs	Long-thinking runs	Median thinking tokens	Min	Max
Account 1	40	0/40	968	463	1295
Account 3	11	0/11	716	595	1316
Account 4	385	384/385	7709	3742	31604

Task family	Active account	Runs	Median output tokens	Min	Max
bench-printf-format	Account 1	337	4645	3789	6595
bench-printf-format	Account 3	26	4225	3726	5181
bench-printf-format	Account 4	152	25144	16212	43700
stamp-interpreter	Account 1	590	1716	1357	2232
stamp-interpreter	Account 4	313	7310	4494	16896

Single-seat runs (arm A, arm AH, and the constituent attempts of arm K), joined by start time to a sanitized extract of the maintainer’s account-switcher log (switch times and account slot numbers only). 5 of 1947 runs straddle a switch and are left out. Top: runs that recorded thinking tokens; long thinking means at least 4000 thinking tokens, the threshold fixed in the Study 2 pre-registration. Bottom: output tokens, which every run recorded, for the two Study 2 task families. Account numbers are slots in the switcher, not identifiers.

to A was 8.56 times on **stamp-interpreter**, 6.16 times on **stamp-2**, and 18.17 times on **printf**. K and C cost respectively \$0.637 and \$0.602 per correct answer on **stamp-interpreter**, \$0.545 and \$0.595 on **stamp-2**, and \$3.87 and \$4.97 on **printf**. These are list-price calculations under the recorded accounts and include all K attempts, including those not selected.

4.2 Account-associated thinking

Table 2 joins single-seat run times to the sanitized switch log. Among runs with thinking telemetry, account 4 had 384/385 at or above 4,000 thinking tokens. Accounts 1 and 3 had none among 51 combined runs. Median thinking was 7,709 tokens on account 4, against 968 and 716 on accounts 1 and 3. Within each of the two task families shown, account 4’s output-token range lay entirely above those of the other measured accounts. These are strong associations under the same alias and settings, not evidence that the client can identify what the provider changed.

The association matters for interpreting accuracy. Short-thinking single-agent measurements on

Table 3: Study 2 descriptive outcomes by task, stratum and arm. Inference excludes mixed-account seeds and printf seed 501; the eligible long-thinking comparisons use fewer seeds than some descriptive rows.

Descriptive rows retain printf seed 501; it is excluded from both primary and exploratory comparisons.						
Family	Stratum	A	AH	B	K	C
stamp-interpreter	long thinking	18/19	19/19	19/19	19/19	19/19
stamp-interpreter	mixed accounts	1/1	1/1	1/1	1/1	1/1
bench-printf-format	short thinking	0/1	0/1	1/1	0/1	0/1
bench-printf-format	long thinking	12/16	12/16	11/16	13/16	10/16
bench-printf-format	mixed accounts	0/1	0/1	1/1	1/1	0/1

stamp-interpreter came from several cohorts: the admission pilot, the Study 1 A grid, constituent K attempts, a cap control, and the window control. Their estimates ranged from 0.475 to 0.825. The Study 1 grid’s 33/40 was therefore one estimate within an observed span, not a stable model parameter. The separate long-thinking window-control cohort had 78/80 successful constituent single attempts. Study 2’s long-thinking A arm passed 18/19 on that task and cost \$0.136 per correct answer. These are different cohorts, not a randomized account comparison; their gap does not show that either account or effort caused an accuracy change. The source of the variation among short-thinking estimates is also unidentified. **stamp-2** was not measured under long thinking, so the long-thinking interpreter result is confined to **stamp-interpreter**.

4.3 Study 2: the primary stratum was nearly absent

Study 2’s pre-registered primary used short-thinking seeds. It obtained no eligible short-thinking **stamp-interpreter** seed and only one printf seed. The interpreter’s apparent short-thinking seed crossed accounts between arms and was excluded from within-regime inference. One printf seed also crossed accounts. Table 3 retains those rows descriptively; the printf seed with a cap deviation is likewise descriptive only. The missing ten cells and excluded seeds leave the registered primary comparisons essentially unobserved.

Unlaunched cells are absent from the denominators; launched cells with non-pass outcomes count as failures. Unknown cost is not treated as zero.

Within the exploratory long-thinking stratum, the interpreter comparisons use 19 seeds per arm. A passed 18/19; AH, B, K and C each passed 19/19. On printf, the inferential sample has 15 seeds per arm: A and AH passed 11/15, B and C 10/15, and K 12/15. Every Holm-adjusted comparison against A had $p = 1.000$. That test result means no arm was distinguished in these small samples, not that their effects are equal or that the primary prediction was confirmed. On the interpreter, B cost \$0.269 per correct answer and C \$1.407, with both passing 19/19. The pair’s observed cost was about a fifth of the chatroom’s. The pre-registered cost prediction concerned short thinking; this exploratory long-thinking inequality does not confirm it.

4.4 How seats check each other

This subsection is an exploratory audit added after both studies and was not pre-registered. It asks what one seat’s check of another seat’s work consisted of. It covers every room the hub recorded between 18 and 23 September 2026 in which seats posted structured verification entries: 28 rooms,

Table 4: What each structured verification entry ran, by the kind of project its room worked on.

What the verifying command ran	Hub	Web app	Other	All
Build, type-check or the project’s existing tests	45	121	3	169
Scripted smoke client against a built hub (no model seats)	56	0	0	56
A check the verifying seat wrote itself	4	2	0	6
The built application driven in a real browser	0	12	0	12
Model-driven agents run on the changed build	0	0	0	0
Rooms	16	10	2	28

Every structured verify entry recorded by the hub (243 entries), classified by what its command ran. Hub rooms changed this system; web-app rooms built a browser game. Commands matching a browser, probe or agent pattern (28) were classified by hand; the rest by whether they ran the smoke script. 235 of 243 entries report exit code 0 and 117 of 243 name the commit they checked.

of which 16 changed this system, 10 built a browser game and 2 worked on another codebase. These rooms used a mix of Claude, Codex and DeepSeek models; they are not the benchmark runs of Studies 1 and 2.

What the rule asks for. A verification entry satisfies the gate when it names the proposal, comes from a connection other than the proposer’s, and reports exit code 0. The rule does not ask what the command exercised or whether it differs from the author’s own check, and an entry that reports a failure is not a verdict: only a passing entry counts. When an author proposes without verification, the hub’s refusal tells them to post the command they ran and adds that “someone else must then run it”. While a proposal is open, the hub’s prompt calls it “open and untested” until some seat names its weakest claim in one sentence. By construction, then, verification certifies that an author’s evidence reproduces. Nothing plays the part of the held-out acceptance tests that a solver never sees in SWE-bench [3].

What verification entries ran. Table 4 and Figure 4a classify all 243 structured entries by what their command ran. Of these, 235 reported exit code 0 and 117 named the commit they checked. None started model-driven agents on a changed build, including every entry from the 16 rooms that changed this system. Twelve drove a built web application in a real browser, all in game rooms, and six ran a check the verifying seat had written. The other 225 ran a build, a type check, the project’s existing tests, or a scripted smoke client that drives a built hub without model seats. The 103 entries whose proposal author is known and differs from the verifier show the same pattern: 51/103 ran existing tests and 41/103 the smoke client.

What review acts found. In four of these rooms, two that changed this system and two game rooms, an AI auditor catalogued every review act from the room log and each seat’s own transcript. A second auditor re-counted each room and tried to refute the first (Table 5, Figure 4b). Re-running the author’s test, suite or build was the most common method, 154 of 331 acts, and the least likely to find a defect: 39/154, against 42/73 for reading the diff and 25/38 for a check the author had not supplied. These shares are associations, not effects of method; a reviewer may write its own check because it already suspects a problem.

In both rooms that changed this system, neither auditor found a seat other than a change’s author running agents on a changed build. The acts that the first auditor counted as using the system “as its users would” ran the scripted smoke client or observed the unchanged live hub. In the first of these rooms, seats asked about 35 times whether work had been tested; nearly every request asked for a commit and a command to re-run, not for evidence that the change worked

Table 5: Review acts by method in the four audited rooms, and how many found a defect.

Review method	Hub A	Hub B	Game 1	Game 2	All	Found a defect
Re-ran author’s test/build	44	12	59	39	154	39
Read the diff only	20	18	21	14	73	42
Own check	13	7	9	9	38	25
Used it as users would	3	12	8	14	37	14
Checked commit contents	6	1	3	11	21	11
No stated check	3	2	3	0	8	0
All review acts	89	52	103	87	331	131

Review acts (one seat checking another’s work) in two rooms that changed this system and two that built a browser game, classified by an AI auditor from the room logs and the seats’ own transcripts. A second auditor re-counted each room; it moved individual acts between methods but confirmed the main pattern (Section 4.4).

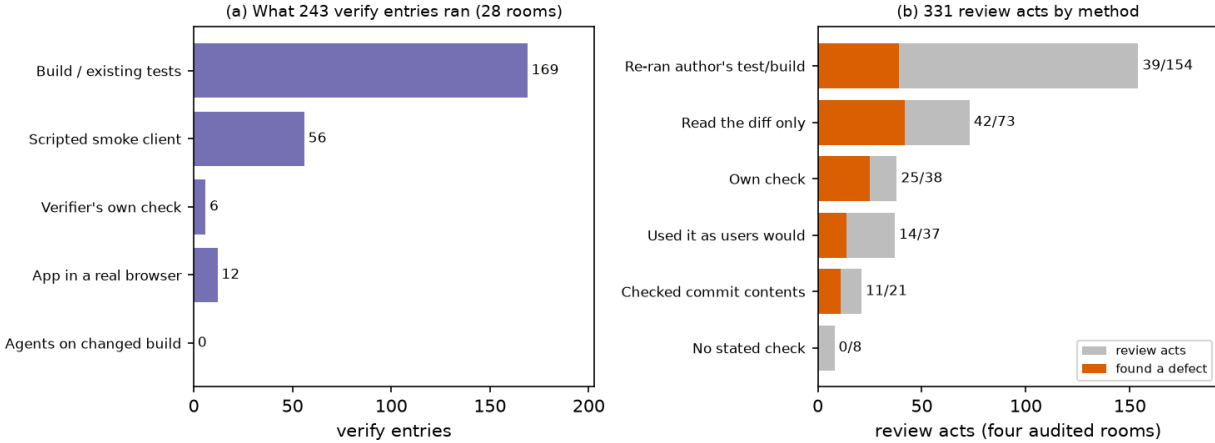


Figure 4: (a) What all 243 structured verification entries ran. (b) Review acts in the four audited rooms by method; the orange share found a defect.

for agents. In the game rooms, non-authors did open the game, at least 8 and 15 times by the auditors’ counts. Even so, the first auditor listed 12 and 10 merged changes that no non-author had exercised in the game; the re-counts disputed individual items but not the pattern. In the first game room, reviewers had seen one change only in night scenes, and it failed for the owner in daylight six minutes after it merged.

Reading. Across both kinds of project and both measures, seats checked each other often, but a check usually meant reproducing the author’s evidence, which is what the verification rule asks for. Defects came disproportionately from reading the diff and from checks a reviewer built. The audit identifies a property of the design, not a causal effect: no room ran under a different verification rule.

5 What this does not show

The planned vote-versus-chatroom comparison under one stable serving condition was never run at adequate power.

The maintainer’s working pipeline—prompt, prior-art agent, plan, build, and devkit review—has never been compared with one agent, a builder-reviewer pair, or a room. The marginal value

of review by a non-author was not identified either. B ran in Study 2 and in a small pilot, but the Study 2 primary was essentially absent. A separate build-suite diagnostic reached a 9/9 ceiling in every cell.

The bundled chatroom build was compared with other arms, but its individual mechanics were not ablated. Regression tests show that gates and assignments operate as coded; one bundled-build trial is not a causal estimate for a gate. No effect size is assigned here to blind openings, challenge, amended proposals, quote-checked votes, reviewer assignment, or verification.

The review audit (Section 4.4) is post hoc and was not pre-registered. Its first measure classifies commands by pattern; every command that matched a browser, probe or agent pattern was labelled by hand by one annotator, an AI assistant working for the maintainer. Its second measure rests on AI auditors whose counts moved by several acts on re-count. Its rooms mix models and projects and were run by one operator. It shows what verification consisted of, not what a different rule would have caught.

Other engineering observations are outside the task-success comparison. Held waits were measured at 115.6 million versus 53.6 million tokens in one room each on different tasks, so that contrast is observational. OpenRouter sticky-session caching was mock-tested only. Checkpoint trimming has no answer-quality test. None of these results supports a claim about the measured A, B, C, or K accuracy differences.

6 Related work

Multi-agent debate provides the immediate setting for this comparison. Du et al. [1] study debate as a route to better answers, while Smit et al. [6] examine how debate strategies compare with simpler baselines. Those papers motivate asking whether a conversational protocol earns its additional cost. Our measured question is narrower: on three fixed machine-scored tasks, did this recorded chatroom differ from a vote over fresh attempts?

Arm K’s interpreter selector is a form of self-consistency: independently generated answers are normalized and the most frequent is submitted [8]. Its code selector instead uses execution agreement among candidate programs, following the type of signal studied by Shi et al. [5]. In the observed printf pool, agreement favored the common wrong implementation. That is why the candidate-availability ceiling and the submitted-answer rate must be reported separately. The result describes the selector as used here; it is not a comparison among alternative selectors.

The chatroom’s independent openings, challenge, proposal and verification steps make it tempting to attribute corrected answers to a particular interaction. Work on divergent thinking and premature consensus [4; 9] gives reasons to inspect those interactions. Work on minority answers in multi-agent debate [2] likewise frames the wrong-cluster observation as a testable question rather than an established general rule.

Finally, the hidden machine oracle keeps the agents’ own consensus separate from correctness. Research on language-model judges [10; 7] concerns a different measurement route; none of the pass rates here uses such a judge. Our main measurement complication arose earlier, at model serving: the same recorded alias and settings accompanied different thinking-token distributions on different active accounts. The account log and token telemetry therefore belong alongside accuracy and cost in any attempt to reproduce these particular comparisons.

A Pre-registration, deviations and reproducibility

What was fixed in advance. The Study 1 A–C comparison and its per-task test were specified before any Study 1 run. The vote arm (K), its group sizes, selectors, per-attempt caps and six-test Holm family were specified after the A and C results but before any K run. Study 2’s five arms, seeds, rotation of arm order within each seed, thinking-token threshold, fourteen-test Holm family and predictions were specified before any Study 2 cell existed.

Recorded changes. Before the Study 1 grid, the original task suite was replaced following screening, and the long-brief task was retired. The admission rule was tightened after earlier candidates proved near ceiling; the below-0.3 exclusion category was added after observing printf v2. The retained printf pilot and post-data version selection are described in Section 3. The original fifteen-seed design with sequential stopping was replaced by forty seeds per task. Dollar and wall-clock ceilings derived from each paired C run replaced token/turn matching; seeds served as bookkeeping labels without controlling sampling or seat assignment. The room size was fixed at three, the default deadline increased from 300 to 900 seconds, turn counts came directly from response JSON, and deadline-killed runs were classified as timeouts and made retryable. K’s caps, selectors and six-test family were settled before K collection. The later A–C analysis revision is described in Section 3. The auxiliary same-window control was stopped after three runs for cost and resumed at reduced scope.

In Study 2, the printf deadline bound on its first A cell. Caps were then raised for A, AH and every K attempt, and the printf block’s spending ceiling increased from 74 to 240 USD. The original timed-out A cell was retained separately and rerun; seed 501’s inferential exclusion is described in Section 3. Killed cells with partial usage from every seat subsequently counted toward the launch budget at list-price estimates multiplied by 1.5; this did not recover their actual cost. An outcome-blind rule invalidated nine quota-refused cells and the operator-killed B cell at printf seed 517; all were rerun, retaining their known spend. Remaining launches were restricted to account 4. Account-log stratification, mixed-seed exclusions and the post-data long-thinking tests are described in Sections 3 and 4.3. The switch-log extract was later refreshed to correct resumed-cell assignments. The maintainer stopped after printf seed 518 because further cells were judged unlikely to change the observed lack of differences; this was an outcome-informed stop without a pre-specified stopping rule.

How the numbers were produced. Every table and figure is generated by scripts from per-run result records; none is edited by hand, and a regeneration check fails if a regenerated table or figure differs from the one included. Account assignment joins each run’s start and end times to a sanitized extract of the account switcher’s log that holds only switch times and account slot numbers. The manual room-correction tallies are described in Section 4.1. The automated audit checks extracted prose decimals, fractions and percentages for numerical matches in generated outputs and listed records, allowing rounding and complementary counts. It omits bare integers and text inside several formatting and link commands, and does not establish that a numerical match supports the surrounding claim.

Availability. The system source, task fixtures, per-run result records, generation scripts, pre-registrations and dated deviation record are archived with the project repository at commit 28961b2 and the manuscript branch built on it.

Author and AI disclosure

Author. [Maintainer name, affiliation, and contact to be filled before submission.]

AI disclosure. Language-model agents helped build the evaluated system, run its recorded experiments, and draft this manuscript. The human author is responsible for checking the evidence, citations, and final submitted text. No agent is listed as an author.

References

- [1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- [2] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in multi-agent llm debates, 2026. URL <https://arxiv.org/abs/2606.29270>.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [4] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate, 2023. URL <https://arxiv.org/abs/2305.19118>.
- [5] Freda Shi, Daniel Fried, Marjan Ghazvininejad, Luke Zettlemoyer, and Sida I. Wang. Natural language to code translation with execution, 2022. URL <https://arxiv.org/abs/2204.11454>.
- [6] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D. Barrett, and Arnu Pretorius. Should we be going mad? a look at multi-agent debate strategies for llms, 2023. URL <https://arxiv.org/abs/2311.17371>.
- [7] Minsoo Song, Chanwoo Kim, Sugyeong Eo, and Chanjun Park. Beyond consensus: Downward bias and role asymmetry in multi-agent llm judges for subjective evaluation, 2026. URL <https://arxiv.org/abs/2608.30373>.
- [8] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- [9] Binwei Yao, Chao Shang, Wanyu Du, Jianfeng He, Ruixue Lian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Peacemaker or troublemaker: How sycophancy shapes multi-agent debate, 2025. URL <https://arxiv.org/abs/2509.23055>.
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.